

DEPARTAMENTO:	Ciencias de la computación	CARRERA:	Software		
ASIGNATURA:	Pruebas de software	NIVEL:	6to	FECHA:	04/12/2025
DOCENTE:	Ing. LUIS ALBERTO CASTILLO SALINAS	PRÁCTICA N°:	1	CALIFICACIÓN:	

Manejo de pruebas unitarias

Moisés Sebastián Benalcázar Farinango

RESUMEN

Se implementaron pruebas unitarias en una aplicación Angular siguiendo el enfoque Test-Driven Development (TDD). Inicialmente se crearon pruebas manuales en JavaScript puro para una clase Calculator con métodos de multiplicación y división, aplicando el ciclo Red-Green-Refactor y manejando casos especiales como la división por cero. Posteriormente, se configuró y ejecutó el entorno automático de pruebas utilizando Karma y Jasmine, herramientas integradas por defecto en proyectos Angular. Se exploraron diferentes comandos (ng test, --watch, --code-coverage) y se generaron reportes de cobertura de código que alcanzaron el 100 %. Finalmente, se realizaron pruebas sobre componentes Angular reales, verificando la existencia y atributos de elementos del DOM mediante matchers de Jasmine y funciones como querySelector y querySelectorAll. Los resultados demuestran la importancia de las pruebas unitarias para garantizar la calidad y mantenibilidad del código.

Palabras Claves: Pruebas unitarias, Angular, TDD, Karma, Jasmine.

1. INTRODUCCIÓN:

En esta actividad, se crearán pruebas unitarias manejadas en el framework Angular, las mismas serán establecidas de forma manual y posteriormente se desarrollarán pruebas unitarias mediante el uso de las herramientas Karma y Jasmine. Estas herramientas son compatibles con Angular y nos permiten ejecutar pruebas y desplegar un reporte de cobertura de estas.

2. OBJETIVO(S):

- 2.1 Manejar el ciclo de vida de TDD.
- 2.2 Manejar la sintaxis para crear pruebas unitarias en Angular.
- 2.3 Usar el mantra de las 3 As para crear una buena prueba unitaria.

3. MARCO TEÓRICO:

Test-Driven Development (TDD) es una metodología de desarrollo donde primero se escribe una prueba que falla (Red), luego el código mínimo necesario para que pase (Green) y finalmente se refactoriza el código manteniendo las pruebas exitosas (Refactor). Su mantra es "Red-Green-Refactor".

Pruebas unitarias verifican el comportamiento correcto de la unidad más pequeña de código (función, método o clase) de forma aislada.

Jasmine es un framework de pruebas de comportamiento (BDD) para JavaScript. Proporciona funciones como describe(), it(), expect() y matchers (toBe, toEqual, toBeTruthy, etc.). Es el framework por defecto que usa Angular CLI.

Karma: Test runner desarrollado por el equipo de Angular. Ejecuta las pruebas en navegadores reales o headless y muestra los resultados en tiempo real. Permite opciones como --watch y generación de reportes de cobertura.

Angular Testing Utilities: Angular proporciona utilidades como TestBed para configurar y crear componentes en entornos de prueba, además de métodos como component.fixture.detectChanges(), querySelector y debugElement.query.

Cobertura de código: Métrica que indica qué porcentaje del código fuente está siendo ejecutado por las pruebas. En Angular se genera con el flag --code-coverage y se visualiza en la carpeta coverage/.

4. DESCRIPCIÓN DEL PROCEDIMIENTO:

PARTE 1: Establecer el ambiente de desarrollo

Paso 1: Crear un proyecto nuevo de Angular.

- a) Ejecutar el comando `ng new nombreProyecto`, para crear el nuevo proyecto

```
PS C:\Users\Niño Moi\OneDrive\Escritorio\Laboratorio-PruebasS> ng new PruebasUnitarias

Would you like to share pseudonymous usage data about this project with the Angular Team
at Google under Google's Privacy Policy at https://policies.google.com/privacy. For more
details and how to change this setting, see https://angular.dev/cli/analytics.

(y/N)
```

Ilustración 1: Realizamos la creación en una carpeta la cual se tiene un fácil acceso, luego se crea un proyecto en Angular con los comandos 'ng new' seguido del nombre del proyecto.

- b) No manejamos ruteo y manejaremos únicamente CSS

```
Global setting: disabled
Local setting: No local workspace configuration file.
Effective status: disabled
✓ Which stylesheet format would you like to use? CSS [ https://developer.mozilla.org/docs/Web/CSS ]
✓ Do you want to enable Server-Side Rendering (SSR) and Static Site Generation (SSG/Prerendering)? Yes
✓ Do you want to create a 'zoneless' application without zone.js? No
✓ Which AI tools do you want to configure with Angular best practices? https://angular.dev/ai/develop-with-ai None
CREATE PruebasUnitarias/angular.json (2678 bytes)
CREATE PruebasUnitarias/package.json (1388 bytes)
CREATE PruebasUnitarias/README.md (1539 bytes)
CREATE PruebasUnitarias/tsconfig.json (1026 bytes)
CREATE PruebasUnitarias/.editorconfig (331 bytes)
CREATE PruebasUnitarias/.gitignore (647 bytes)
CREATE PruebasUnitarias/tsconfig.app.json (464 bytes)
CREATE PruebasUnitarias/tsconfig.spec.json (449 bytes)
CREATE PruebasUnitarias/.vscode/extensions.json (134 bytes)
CREATE PruebasUnitarias/.vscode/launch.json (490 bytes)
CREATE PruebasUnitarias/.vscode/tasks.json (980 bytes)
CREATE PruebasUnitarias/src/main.ts (228 bytes)
CREATE PruebasUnitarias/src/index.html (315 bytes)
CREATE PruebasUnitarias/src/styles.css (81 bytes)
CREATE PruebasUnitarias/src/main.server.ts (300 bytes)
CREATE PruebasUnitarias/src/server.ts (1677 bytes)
CREATE PruebasUnitarias/src/app/app.spec.ts (697 bytes)
CREATE PruebasUnitarias/src/app/app.ts (310 bytes)
```

Ilustración 2: Damos paso a la instalación únicamente los paquetes para CSS nada más y continuamos.

Paso 2: Revisión de la creación del proyecto.

- a) Abrir el proyecto en el editor de texto de preferencia

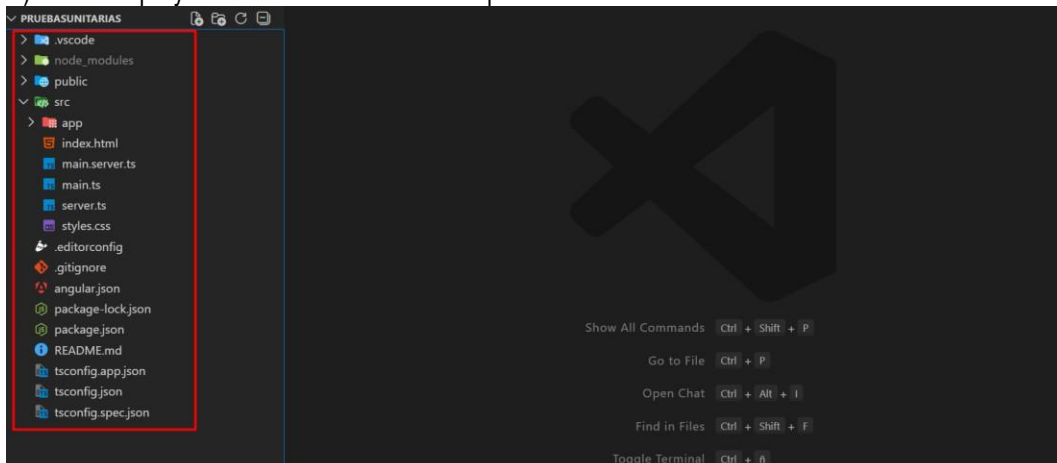


Ilustración 3: Comprobamos la estructura correcta del proyecto mediante el editor de texto Visual Studio Code para probar el proyecto.

- b) Ejecutar el proyecto con el comando `ng s -o`

```
PS C:\Users\Niño Moi\OneDrive\Escritorio\Lab 1\PruebasUnitarias> ng serve -o

Browser bundles
Initial chunk files | Names          | Raw size
main.js             | main           | 48.60 kB
polyfills.js        | polyfills      | 95 bytes
styles.css           | styles         | 95 bytes
                    | Initial total  | 48.79 kB

Server bundles
Initial chunk files | Names          | Raw size
main.server.mjs     | main.server    | 49.87 kB
server.mjs          | server         | 1.81 kB
polyfills.server.mjs | polyfills.server | 266 bytes

Application bundle generation complete. [2.791 seconds] - 2025-12-05T23:06:40.334Z

Watch mode enabled. Watching for file changes...
NOTE: Raw file sizes do not reflect development server per-request transformations.
  → Local:  http://localhost:4200/
  → press h + enter to show help
```

Ilustración 4: Inicializamos el proyecto mediante los comandos 'ng s -o' ó 'ng serve -o' y luego nos aparecera en nuestro navegador de preferencia.

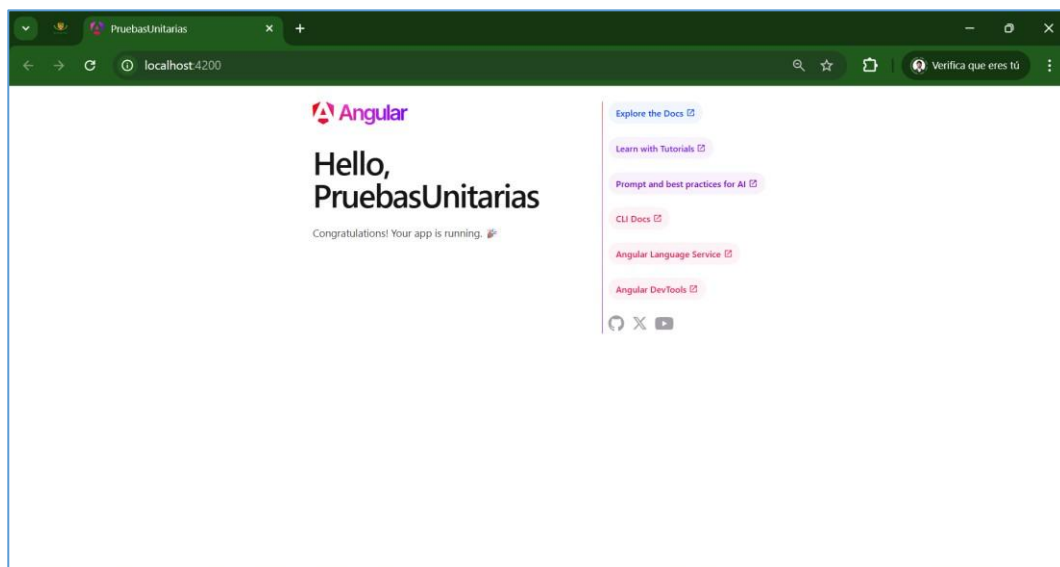
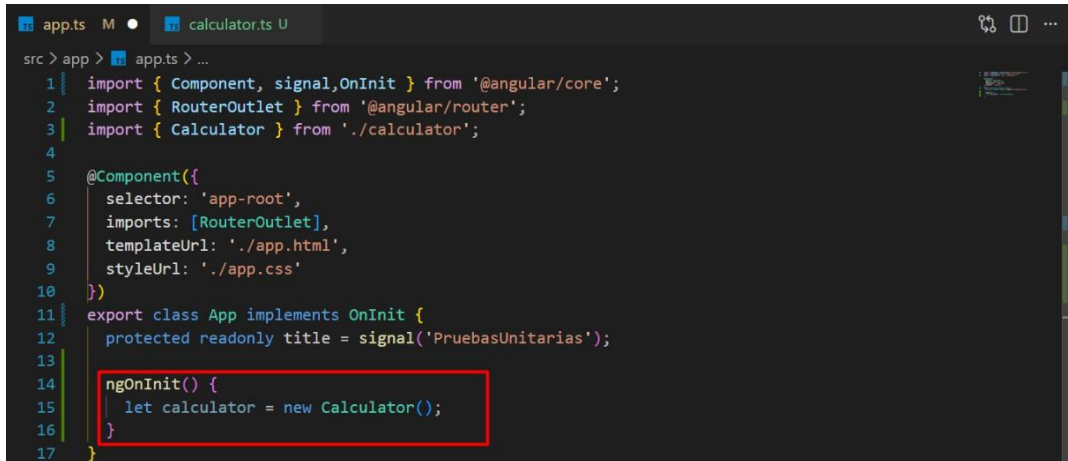


Ilustración 5: Podemos observar el inicio exitoso de la aplicación creada en Angular.

PARTE 2: Establecimiento de pruebas unitarias de forma manual con JavaScript guiadas por TDD

Paso 1: Crear la prueba.

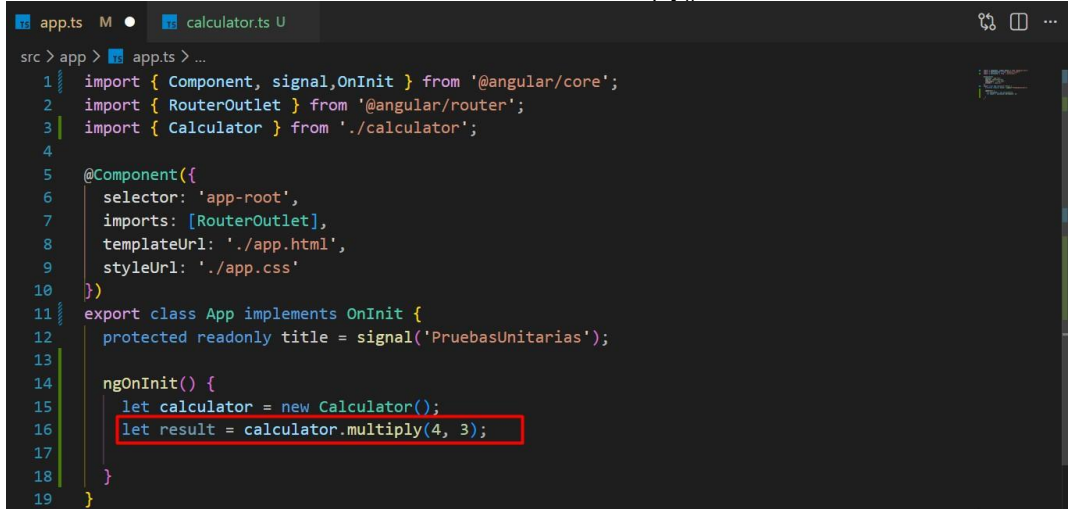
- a. Crear una prueba que defina un objeto de una clase Calculator()



```
src > app > app.ts > ...
1 import { Component, signal, OnInit } from '@angular/core';
2 import { RouterOutlet } from '@angular/router';
3 import { Calculator } from './calculator';
4
5 @Component({
6   selector: 'app-root',
7   imports: [RouterOutlet],
8   templateUrl: './app.html',
9   styleUrls: ['./app.css']
10 })
11 export class App implements OnInit {
12   protected readonly title = signal('PruebasUnitarias');
13
14   ngOnInit() {
15     let calculator = new Calculator();
16   }
17 }
```

Ilustración 6: definimos un objeto de la clase 'Calculator()' en una variable con nombre similar.

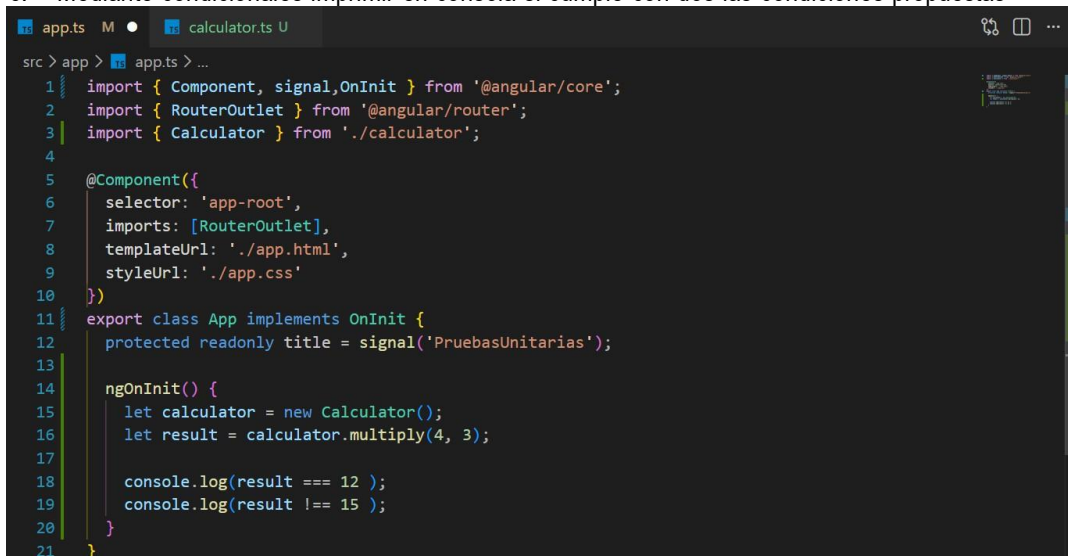
- b. En una variable almacenar el resultado del método multiply() de la clase anteriormente creada



```
src > app > app.ts > ...
1 import { Component, signal, OnInit } from '@angular/core';
2 import { RouterOutlet } from '@angular/router';
3 import { Calculator } from './calculator';
4
5 @Component({
6   selector: 'app-root',
7   imports: [RouterOutlet],
8   templateUrl: './app.html',
9   styleUrls: ['./app.css']
10 })
11 export class App implements OnInit {
12   protected readonly title = signal('PruebasUnitarias');
13
14   ngOnInit() {
15     let calculator = new Calculator();
16     let result = calculator.multiply(4, 3);
17   }
18 }
19 }
```

Ilustración 7: almacenamos el resultado en una variable de la clase 'Calculator()', para poder comprobar mediante las pruebas a continuación.

- c. Mediante condicionales imprimir en consola si cumple con dos las condiciones propuestas



```
src > app > app.ts > ...
1 import { Component, signal, OnInit } from '@angular/core';
2 import { RouterOutlet } from '@angular/router';
3 import { Calculator } from './calculator';
4
5 @Component({
6   selector: 'app-root',
7   imports: [RouterOutlet],
8   templateUrl: './app.html',
9   styleUrls: ['./app.css']
10 })
11 export class App implements OnInit {
12   protected readonly title = signal('PruebasUnitarias');
13
14   ngOnInit() {
15     let calculator = new Calculator();
16     let result = calculator.multiply(4, 3);
17
18     console.log(result === 12 );
19     console.log(result !== 15 );
20   }
21 }
```

Ilustración 8: probar mediante condiciones si cumple o no cumple dos resultados he imprimimos en consola.

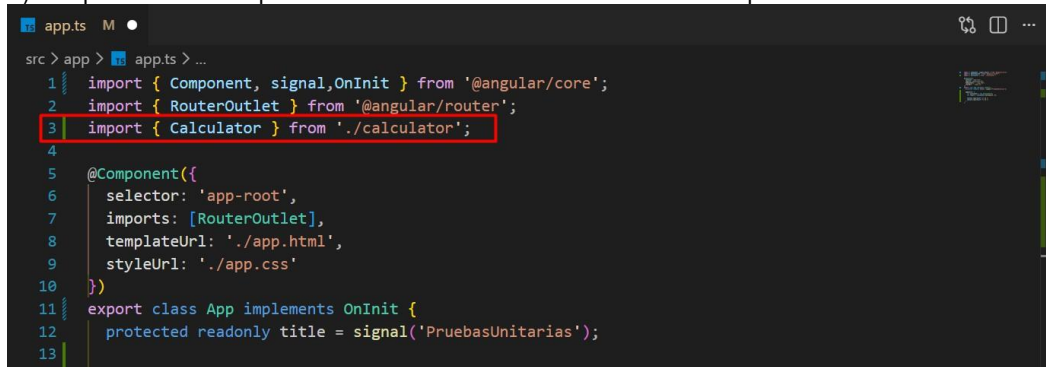
Paso 2: Crear el mínimo código para que la prueba pase o falle.

a) Crear una clase Calculator(), lo podemos hacer con el comando `ng g cl calculator`

```
PS C:\Users\Niño Moi\OneDrive\Escritorio\Lab 1\PruebasUnitarias> ng g cl calculator
CREATE src/app/calculator.spec.ts (177 bytes)
CREATE src/app/calculator.ts (30 bytes)
PS C:\Users\Niño Moi\OneDrive\Escritorio\Lab 1\PruebasUnitarias>
```

Ilustración 9: con el siguiente comando 'ng g cl' podemos generar una clase con el nombre que deseemos.

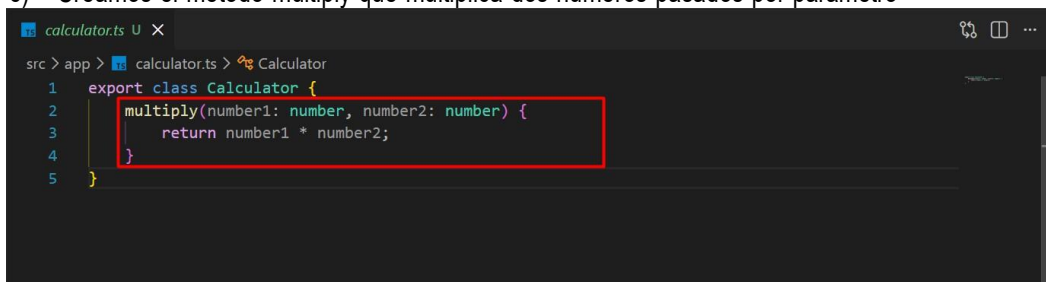
b) Importamos el componente a la clase donde estamos creando la prueba



```
app.ts M
src > app > app.ts > ...
1 import { Component, signal, OnInit } from '@angular/core';
2 import { RouterOutlet } from '@angular/router';
3 import { Calculator } from './calculator';
4
5 @Component({
6   selector: 'app-root',
7   imports: [RouterOutlet],
8   templateUrl: './app.html',
9   styleUrls: ['./app.css']
10 })
11 export class App implements OnInit {
12   protected readonly title = signal('PruebasUnitarias');
13 }
```

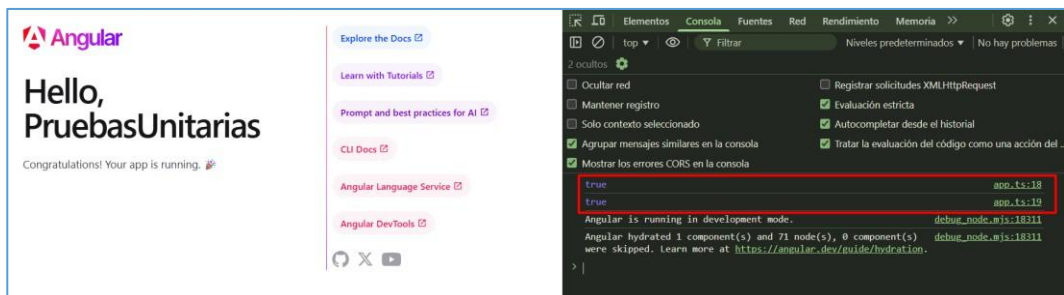
Ilustración 10: importamos la clase para poder tener conexión y realizar exitosamente las pruebas.

c) Creamos el método multiply que multiplica dos números pasados por parámetro



```
calculator.ts U X
src > app > calculator.ts > Calculator
1 export class Calculator {
2   multiply(number1: number, number2: number) {
3     return number1 * number2;
4   }
5 }
```

Ilustración 11: creamos el método que nos permite pasar 2 parámetros que se multiplican entre sí.



The left panel shows the Angular application running in the browser with the message "Hello, PruebasUnitarias". The right panel shows the console output with the following messages:

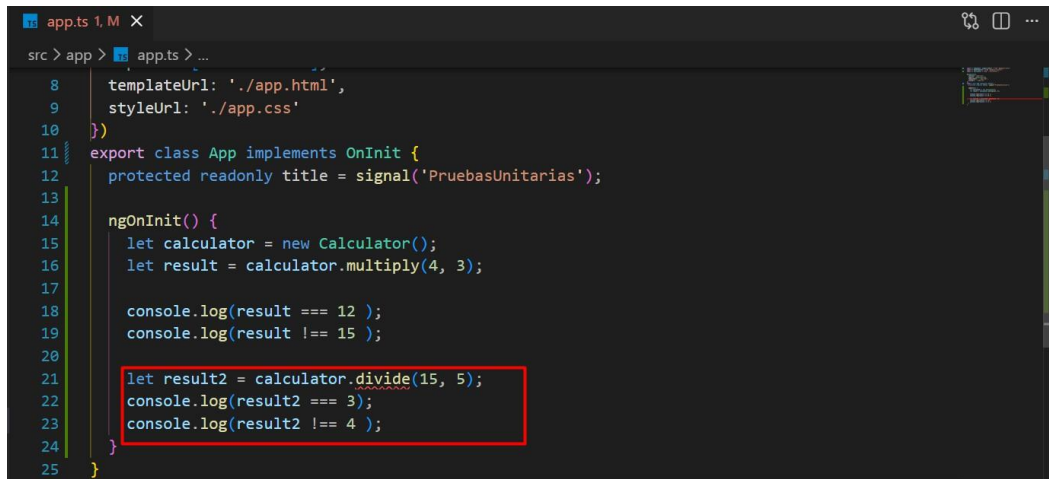
```
Angular
Hello,
PruebasUnitarias
Congratulations! Your app is running.

true app.ts:38
true app.ts:39
Angular is running in development mode. debug_node.mjs:18311
Angular hydrated 1 component(s) and 71 node(s), 0 component(s) debug_node.mjs:18311
were skipped. Learn more at https://angular.dev/guide/hydration.
```

Ilustración 12: observamos dos trues referentes a sus respectivas líneas de las pruebas, pasando exitosamente.

Paso 3: Refactorizar el código.

a) Crear una prueba para un método que permita dividir dos números

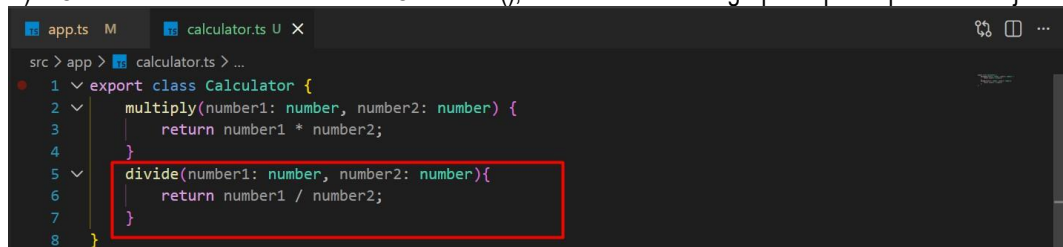


```

src > app > app.ts > ...
8   templateUrl: './app.html',
9   styleUrls: ['./app.css']
10 }
11 export class App implements OnInit {
12   protected readonly title = signal('PruebasUnitarias');
13
14   ngOnInit() {
15     let calculator = new Calculator();
16     let result = calculator.multiply(4, 3);
17
18     console.log(result === 12 );
19     console.log(result !== 15 );
20
21     let result2 = calculator.divide(15, 5);
22     console.log(result2 === 3);
23     console.log(result2 !== 4 );
24   }
25 }
  
```

Ilustración 13: al momento nos da error ya que aun no creamos el código que verifique la función 'divide' para dividir entre dos parámetros.

b) Creamos el método en la clase Calculator(), con el mínimo código para que la prueba se ejecute

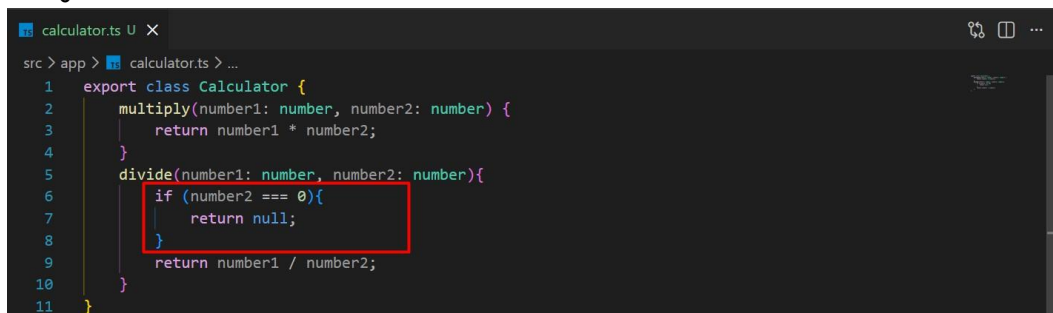


```

src > app > calculator.ts > ...
1 export class Calculator {
2   multiply(number1: number, number2: number) {
3     return number1 * number2;
4   }
5   divide(number1: number, number2: number){
6     return number1 / number2;
7   }
8 }
  
```

Ilustración 14: Creamos el método para que divida entre dos parámetros y realizar la prueba exitosamente.

c) Ahora usaremos la tercera etapa del ciclo de vida TDD que es la refactorización realizando una división para 0



```

src > app > calculator.ts > ...
1 export class Calculator {
2   multiply(number1: number, number2: number) {
3     return number1 * number2;
4   }
5   divide(number1: number, number2: number){
6     if (number2 === 0){
7       return null;
8     }
9     return number1 / number2;
10  }
11 }
  
```

Ilustración 15: refactorizamos la división igual a 0.

d) Entonces lo que se desearía hacer es que al dividir para 0 pues se pueda responder de una manera diferente a este error, en este caso haremos que retorne un null. Y aquí es donde entra el paso de refactorización


```

src > app > app.ts > ...
11 export class App implements OnInit {
12   protected readonly title = signal(' PruebasUnitarias ');
13
14   ngOnInit() {
15     let calculator = new Calculator();
16     let result = calculator.multiply(4, 3);
17
18     console.log(result === 12 );
19     console.log(result !== 15 );
20
21     let result2 = calculator.divide(15, 5);
22     console.log(result2 === 3);
23     console.log(result2 !== 4 );
24
25     let result3 = calculator.divide(10, 0);
26     console.log(result3 === null);
27   }
28 }
29
  
```

Ilustración 16: comprobamos la refactorización mediante un console.log que nos detecte un null.

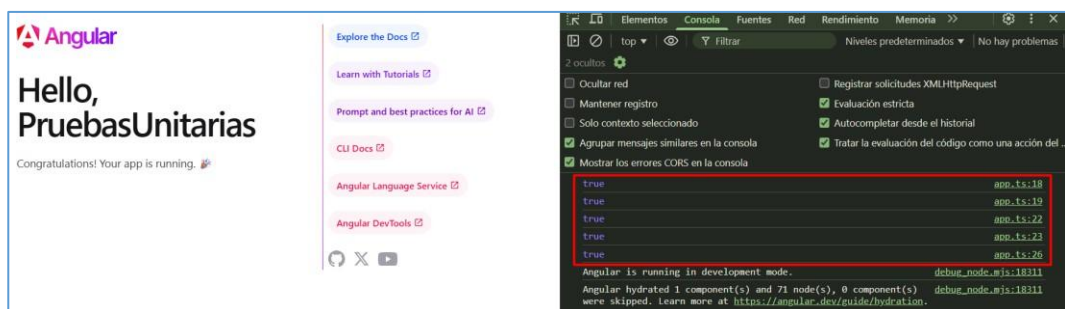


Ilustración 17: comprobamos que ahora tenemos 5 true en consola, indicando unas pruebas exitosas.

PARTE 3: Establecimiento de pruebas unitarias de forma automática con las herramientas Karma y Jasmine

Paso 1: Comandos para ejecutar pruebas en Angular.

a) npm test

```

PS C:\Users\Niño Moi\OneDrive\Escritorio\Lab 1\PruebasUnitarias> npm test

> pruebas-unitarias@0.0.0 test
> ng test
  
```

Ilustración 18: se realiza la ejecución de 'npm tes' para realizar la instalación de cualquier dependencia faltante.

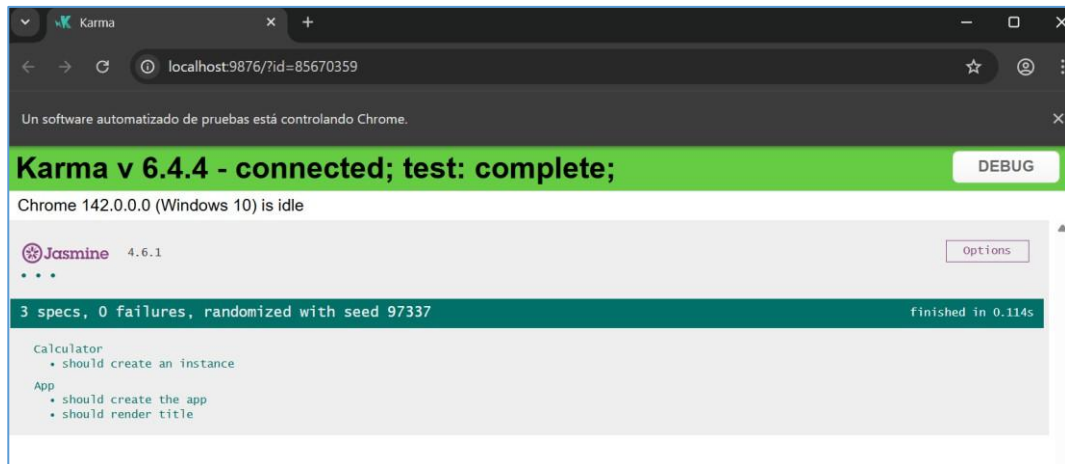


Ilustración 19: se nos abre una nueva ventana de Karma automáticamente en nuestro navegador favorito.

b) ng test

```
PS C:\Users\Niño Moi\OneDrive\Escritorio\Lab 1\PruebasUnitarias> ng test

Initial chunk files | Names | Raw size |
chunk-X32VFMM2.js | - | 2.50 MB |
polyfills.js | polyfills | 1.04 MB |
spec-app.spec.js | spec-app.spec | 231.41 kB |
jasmine-cleanup-1.js | jasmine-cleanup-1 | 67.20 kB |
test_main.js | test_main | 21.79 kB |
chunk-TTULUY32.js | - | 1.99 kB |
jasmine-cleanup-0.js | jasmine-cleanup-0 | 519 bytes |
chunk-BGAI4XN5.js | - | 509 bytes |
spec-calculator.spec.js | spec-calculator.spec | 326 bytes |
styles.css | styles | 95 bytes |

| Initial total | 3.86 MB

Application bundle generation complete. [2.579 seconds] - 2025-12-06T23:56:26.770Z
```

Ilustración 20: se realiza 'ng test' y se obtiene un resultado similar ya que ahora únicamente se ejecuta las pruebas y no instala o limpia ninguna dependencia.

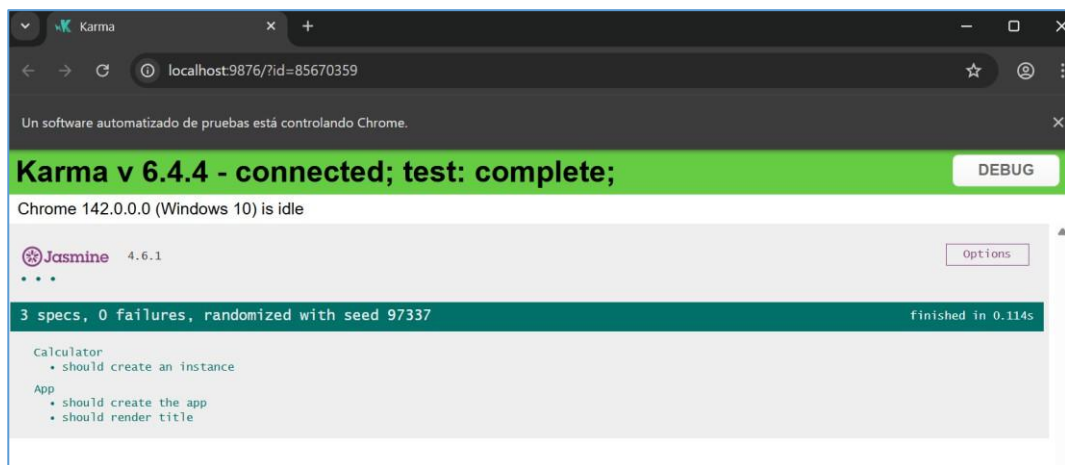


Ilustración 21: nos muestra una nueva ventana la cual también se abre varias veces así cerremos las ventanas hasta que falle la ejecución.

c) ng test --watch=false


```

PS C:\Users\Niño Moi\OneDrive\Escritorio\Lab 1\PruebasUnitarias> ng test --watch=false
Initial chunk files | Names | Raw size |
chunk-X32VFMM2.js | - | 2.50 MB |
polyfills.js | polyfills | 1.04 MB |
spec-app.spec.js | spec-app.spec | 231.41 kB |
jasmine-cleanup-1.js | jasmine-cleanup-1 | 67.20 kB |
test_main.js | test_main | 21.79 kB |
chunk-TTULUY32.js | - | 1.99 kB |
jasmine-cleanup-0.js | jasmine-cleanup-0 | 519 bytes |
chunk-BGAI4XN5.js | - | 509 bytes |
spec-calculator.spec.js | spec-calculator.spec | 326 bytes |
styles.css | styles | 95 bytes |
| Initial total | 3.86 MB

Application bundle generation complete. [2.953 seconds] - 2025-12-06T23:58:49.967Z

06 12 2025 18:58:50.130:INFO [karma-server]: Karma v6.4.4 server started at http://localhost:9876/
06 12 2025 18:58:50.131:INFO [launcher]: Launching browsers Chrome with concurrency unlimited
06 12 2025 18:58:50.148:INFO [launcher]: Starting browser Chrome
06 12 2025 18:58:51.640:INFO [Chrome 142.0.0.0 (Windows 10)]: Connected on socket FNxAB18Z_zJBJ-tKAAAB with id 45952304
LOG: true
  
```

Ilustración 22: ahora con 'ng test --watch=false' se obtiene una ventana que muestra los resultados y luego de ejecutarse se cierra, se ejecuta las pruebas y nos muestra el resultado mediante la línea de comandos.

d) ng test --code-coverage

```

PS C:\Users\Niño Moi\OneDrive\Escritorio\Lab 1\PruebasUnitarias> ng test --code-coverage
ed
06 12 2025 19:03:04.832:INFO [launcher]: Starting browser Chrome
06 12 2025 19:03:06.337:INFO [Chrome 142.0.0.0 (Windows 10)]: Connected on socket 3L-EGV0Kk3arrzu4AAAB with id 5072488
LOG: true
Chrome 142.0.0.0 (Windows 10): Executed 1 of 3 SUCCESS (0 secs / 0.004 secs)
LOG: true
Chrome 142.0.0.0 (Windows 10): Executed 1 of 3 SUCCESS (0 secs / 0.004 secs)
LOG: true
Chrome 142.0.0.0 (Windows 10): Executed 1 of 3 SUCCESS (0 secs / 0.004 secs)
LOG: true
Chrome 142.0.0.0 (Windows 10): Executed 1 of 3 SUCCESS (0 secs / 0.004 secs)
Chrome 142.0.0.0 (Windows 10): Executed 1 of 3 SUCCESS (0 secs / 0.004 secs)
Chrome 142.0.0.0 (Windows 10): Executed 3 of 3 SUCCESS (0.131 secs / 0.117 secs)
TOTAL: 3 SUCCESS

===== Coverage summary =====
Statements : 100% ( 16/16 )
Branches : 100% ( 1/1 )
Functions : 100% ( 3/3 )
Lines : 100% ( 15/15 )
=====
  
```

Ilustración 23: nos muestra la cobertura al final de la ejecución para poder informarnos si cubrimos todo el código o tenemos puntos pendientes.

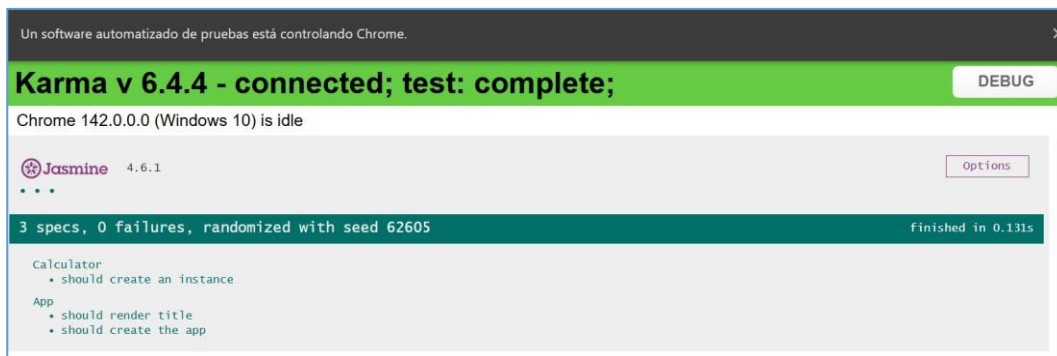


Ilustración 24: nos presenta además una página que no se cerrara hasta que falle la ejecución.

e) `ng test --code-coverage --watch=false`

```
PS C:\Users\Niño Moi\OneDrive\Escritorio\Lab 1\PruebasUnitarias> ng test --code-coverage --watch=false
06 12 2025 19:05:59.801:INFO [launcher]: Starting browser Chrome
06 12 2025 19:06:01.210:INFO [Chrome 142.0.0.0 (Windows 10)]: Connected on socket 3_msyIlgDEGe6LPUDAAAB with id 23748941
LOG: true
Chrome 142.0.0.0 (Windows 10): Executed 2 of 3 SUCCESS (0 secs / 0.068 secs)
LOG: true
Chrome 142.0.0.0 (Windows 10): Executed 2 of 3 SUCCESS (0 secs / 0.068 secs)
LOG: true
Chrome 142.0.0.0 (Windows 10): Executed 2 of 3 SUCCESS (0 secs / 0.068 secs)
LOG: true
Chrome 142.0.0.0 (Windows 10): Executed 2 of 3 SUCCESS (0 secs / 0.068 secs)
LOG: true
Chrome 142.0.0.0 (Windows 10): Executed 2 of 3 SUCCESS (0 secs / 0.068 secs)
Chrome 142.0.0.0 (Windows 10): Executed 3 of 3 SUCCESS (0.088 secs / 0.078 secs)
TOTAL: 3 SUCCESS

===== Coverage summary =====
Statements : 100% ( 16/16 )
Branches   : 100% ( 1/1 )
Functions  : 100% ( 3/3 )
Lines      : 100% ( 15/15 )
=====
PS C:\Users\Niño Moi\OneDrive\Escritorio\Lab 1\PruebasUnitarias>
```

Ilustración 25: unificando los dos comandos obtenemos una única ejecución con el resultado de cobertura.

Paso 2: Archivos de configuración para la ejecución de pruebas.

- Revisión del archivo `karma.conf.js`
- Revisión del archivo `test.ts`
- Revisar el reporte de coverage que se crea, en la carpeta correspondiente, al ejecutar el comando `ng test --code-coverage`

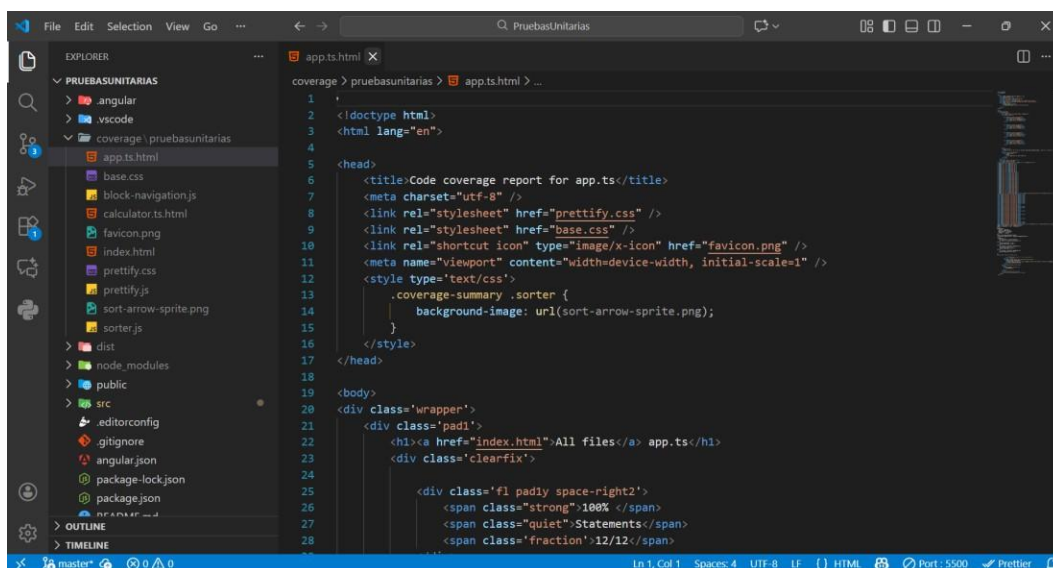


Ilustración 26: localizamos el archivo que nos muestra el reporte de cobertura de nuestra ejecución.

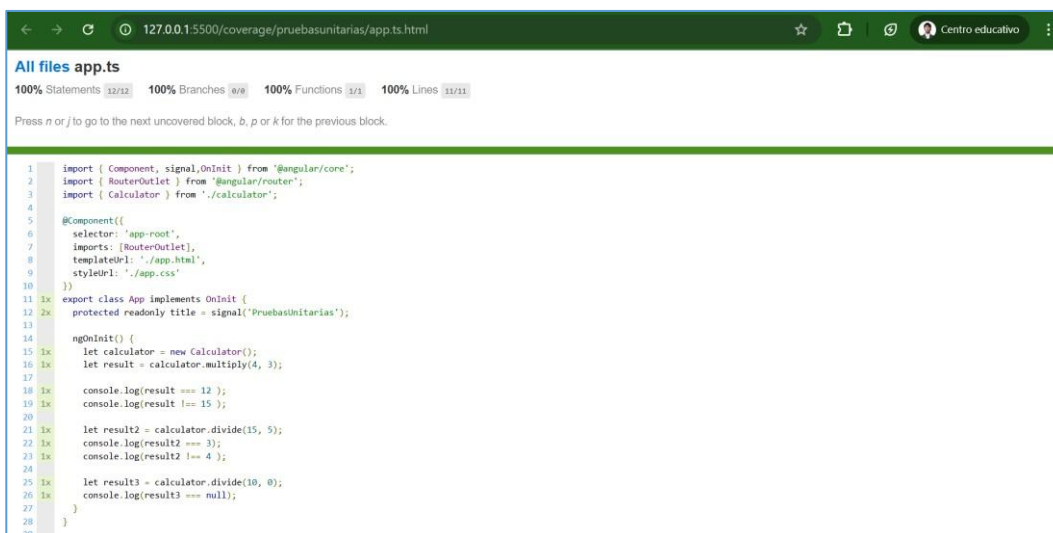


Ilustración 27: al ejecutar se puede observar un resumen de la cobertura al ejecutar las pruebas TDD.

d) Para ejecutar sin estar buscando el archivo en el explorador, es con el comando `http-server coverage/`

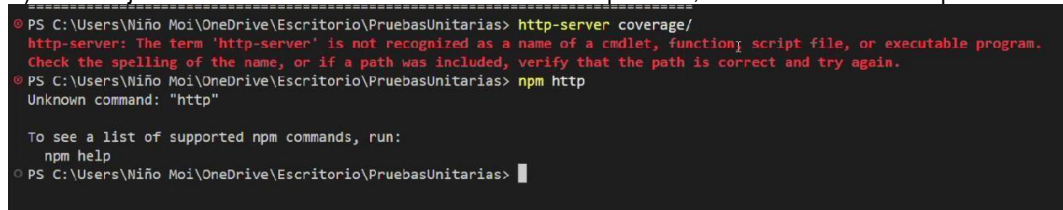


Ilustración 28: se obtuvo un error en la ejecución por que no se encuentra instalada la parte de http-server.

e) Pero para que este funcione debe instalarse `npm install -g http-server`

```
C:\WINDOWS\system32 x + v
Microsoft Windows [Versión 10.0.26100.6899]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\Niño Moi>npm install -g http-server

added 48 packages in 5s

15 packages are looking for funding
  run 'npm fund' for details

C:\Users\Niño Moi>
```

Ilustración 29: realizamos la instalación global de los paquetes y dependencias para poder observar el reporte de cobertura de manera automática.

```
PS C:\Users\Niño Moi\OneDrive\Escritorio\Lab 1\PruebasUnitarias> ng test --code-coverage --
watch=false
=====
PS C:\Users\Niño Moi\OneDrive\Escritorio\Lab 1\PruebasUnitarias> http-server coverage/
Starting up http-server, serving coverage/

http-server version: 14.1.1

http-server settings:
CORS: disabled
Cache: 3600 seconds
Connection Timeout: 120 seconds
Directory Listings: visible
AutoIndex: visible
Serve GZIP Files: false
Serve Brotli Files: false
Default File Extension: none

Available on:
  http://192.168.56.1:8081
  http://192.168.0.7:8081
  http://127.0.0.1:8081
Hit CTRL-C to stop the server
```

Ilustración 30: volvemos a ejecutar y observamos que se tiene una ejecución correcta del código.

```
< → ↺ ⚠ No es seguro 192.168.56.1:8081

Index of /

(drw-rw-rw-) 06-dic-2025 19:02 pruebasunitarias/

Node.js v22.16.0/ http-server server running @ 192.168.56.1:8081
```

Ilustración 31: tenemos que dar clicl sobre los enlaces que nos aparece al final de la ejecución de 'http-server coverage/', luego ingresamos a el nombre de nuestra carpeta, en mi caso 'pruebasunitarias/'.

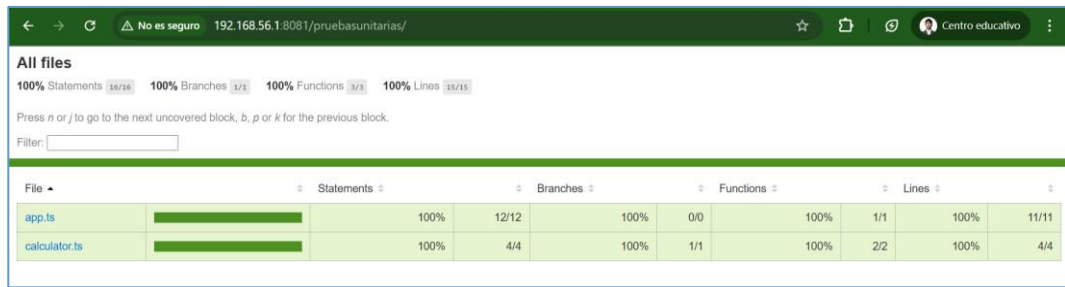


Ilustración 32: finalmente se observa el reporte de cobertura gráficamente y de manera más detallada.

Paso 3: Realización de pruebas con Karma y Jasmine.

- a) Siempre se debe tener en cuenta la sintaxis, en la mayoría de las herramientas se tiene un describe y un it, y se realiza la comparación del valor esperado con el valor real.

```

1 import { Calculator } from './calculator';
2
3 describe('Calculator', () => {
4   describe('Test multiply method', () => {
5     it('should return twelve', () => {
6       // Arrange
7       let calculator = new Calculator();
8       let number1 = 4;
9       let number2 = 3;
10      let expected = 12;
11
12      // Act
13      let result = calculator.multiply(number1, number2);
14
15      // Assert
16      expect(result).toEqual(expected);
17    });
18  });
19 });
  
```

Ilustración 33: estructuramos de manera correcta, con describe para la suite de pruebas y con It para definir las pruebas.

- b) Creamos nuevamente el reporte de cobertura y vemos los resultados arrojados

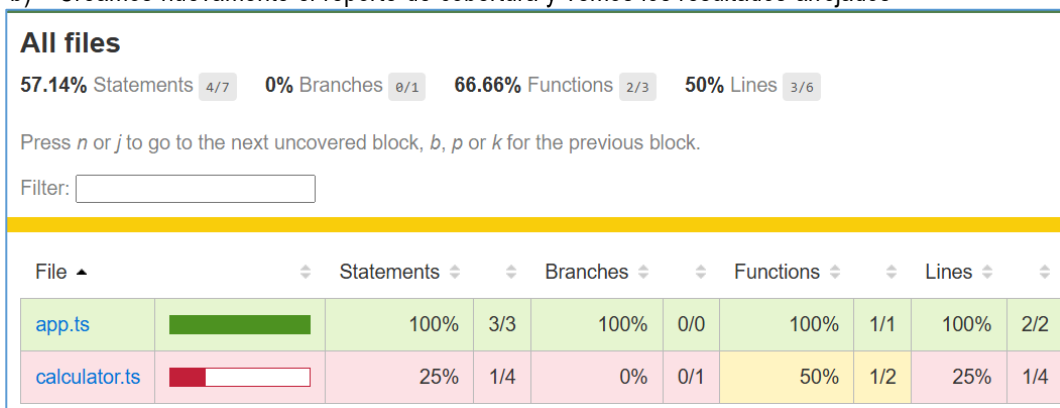


Ilustración 34: realizamos un reporte de cobertura para ver qué porcentaje de código hemos cubierto hasta el momento.

- c) Seguimos ejecutando más casos de prueba, haciendo que las pruebas fallen

```
describe(' Test divide method', () => {
  it('divide for a number', () => {
    // Act & Assert
    expect(calculator.divide(6, 2)).toEqual(3);
    expect(calculator.divide(15, 2)).toEqual(7.5);
  });

  it('divide for zero', () => {
    // Act & Assert
    expect(calculator.divide(20, 0)).toBeNull();
    expect(calculator.divide(155, 0)).toBeNull();
    expect(calculator.divide(561254, 0)).toBeNull();
  });
});
```

Ilustración 35: agregamos la otra parte ahora para probar la división y si condicional divisible para cero.

All files

85.71% Statements 6/7 0% Branches 0/1 100% Functions 3/3 83.33% Lines 5/6

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File		Statements	Branches	Functions	Lines
app.ts		100%	3/3	100%	0/0
calculator.ts		75%	3/4	0%	0/1

Ilustración 36: realizamos el segundo reporte de cobertura alcanzando un 75%, indicando que vamos por buen camino.

All files

100% Statements 7/7 100% Branches 1/1 100% Functions 3/3 100% Lines 6/6

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File		Statements	Branches	Functions	Lines
app.ts		100%	3/3	100%	0/0
calculator.ts		100%	4/4	100%	1/1

Ilustración 37: finalmente se logra alcanzar un 100% mediante el reporte de cobertura.

d) Probar los matcher de Jasmine (se puede ver en su página oficial)

```
describe(' Test Matchers', () => {
  it('test of matchers', () => {
    let name = 'Moises';
    let name2 = '';

    expect(name).toBeDefined();
    expect(name2).toBeUndefined();

    expect(1 + 1 === 2).toBeTruthy();
    expect(1 + 1 === 3).toBeFalsy();

    expect(4).toBeLessThan(19);
    expect(50).toBeGreaterThan(5);

    expect('Evalua cadenas de texto').toContain('/aden/');
    expect(['chair', 'table', 'desk']).toContain('desk');
  });
});
```

Ilustración 38: realizamos las pruebas con los matchers que nos da indicaciones en la documentación de Jasmine

e) Ver el uso de `beforeEach()`

```
// simplificar los Arrange
let calculator: any;

beforeEach(() => {
  calculator = new Calculator();
});
```

Ilustración 39: con ayuda del `beforeEach()` podemos simplificar los arrange.

5. PREGUNTAS/ACTIVIDADES:

Realizar 5 casos de prueba para elementos o componentes Angular (orientada a elementos HTML), guiándose de la documentación oficial de Jasmine.



Registro de Usuario

Completa el formulario para crear tu cuenta

Nombre:

Apellido:

Email:

Edad (opcional):

Contraseña:

Confirmar Contraseña:

☐ Acepto los términos y condiciones

Ilustración 40: se creó un formulario el cual nos permite realizar el registro de un usuario.

```
import { TestBed, ComponentFixture } from '@angular/core/testing';
import { FormsModule } from '@angular/forms';
import { App } from './app';

describe('App - Formulario de Registro', () => {
  let component: App;
  let fixture: ComponentFixture<App>;
  let compiled: HTMLElement;

  beforeEach(async () => {
    await TestBed.configureTestingModule({
      imports: [App, FormsModule],
    }).compileComponents();

    fixture = TestBed.createComponent(App);
    component = fixture.componentInstance;
    compiled = fixture.nativeElement as HTMLElement;
    fixture.detectChanges();
  });

  // verificar que el componente se crea correctamente
  it('debe crear el componente', () => {
    expect(component).toBeTruthy();
  });
});
```

Ilustración 41: en la primera prueba se usa `'expect(toBeTruthy())'` un matcher básico de Jasmine.

```
// verificar que el titulo del formulario se renderiza correctamente
it('debe renderizar el titulo "Registro de Usuario"', () => {
  const titulo = compiled.querySelector('h1');
  expect(titulo).toBeTruthy();
  expect(titulo?.textContent).toContain('Registro de Usuario');
});
```

Ilustración 42: para esta prueba se usa 'querySelector()' para acceder al DOM.

```
// verificar que todos los campos de entrada (inputs) existen en el formulario
it('debe tener todos los campos de entrada requeridos', () => {
  const inputNombre = compiled.querySelector('#nombre') as HTMLInputElement;
  const inputApellido = compiled.querySelector('#apellido') as HTMLInputElement;
  const inputEmail = compiled.querySelector('#email') as HTMLInputElement;
  const inputEdad = compiled.querySelector('#edad') as HTMLInputElement;
  const inputPassword = compiled.querySelector('#password') as HTMLInputElement;
  const inputConfirmPassword = compiled.querySelector('#confirmPassword') as HTMLInputElement;

  expect(inputNombre).toBeTruthy();
  expect(inputApellido).toBeTruthy();
  expect(inputEmail).toBeTruthy();
  expect(inputEdad).toBeTruthy();
  expect(inputPassword).toBeTruthy();
  expect(inputConfirmPassword).toBeTruthy();
});
```

Ilustración 43: se usa múltiples 'querySelector()' para elementos HTML y además usamos 'expect().toBeTruthy()' para verificar existencia de 6 inputs.

```
// verificar que el checkbox de terminos y condiciones existe
it('debe tener un checkbox para aceptar terminos y condiciones', () => {
  const checkbox = compiled.querySelector('#aceptaTerminos') as HTMLInputElement;
  expect(checkbox).toBeTruthy();
  expect(checkbox.type).toBe('checkbox');
});
```

Ilustración 44: se usa 'querySelector()' para el checkbox, también 'expect().toBeTruthy()' y 'expect().toBe()' que son matchers de Jasmine

```
// verificar que los botones de envio y limpiar existen
it('debe tener botones de "Registrarse" y "Limpiar"', () => {
  const botones = compiled.querySelectorAll('button');
  expect(botones.length).toBe(2);

  const botonSubmit = botones[0] as HTMLButtonElement;
  const botonLimpiar = botones[1] as HTMLButtonElement;

  expect(botonSubmit.type).toBe('submit');
  expect(botonSubmit.textContent?.trim()).toBe('Registrarse');
  expect(botonLimpiar.type).toBe('button');
  expect(botonLimpiar.textContent?.trim()).toBe('Limpiar');
});
```

Ilustración 45: para esta prueba se usó 'querySelectorAll()' para obtener múltiples botones, 'expect().toBe()' para verificar tipos y textos, y 'expect().length.toBe()' para contar elementos.

6. CONCLUSIONES:

- Se logró aplicar con éxito las pruebas unitarias en Angular usando correctamente la sintaxis de Jasmine y Karma.
- El alcanzar más del 90% de cobertura en el código o ir aumentando con cada reporte de cobertura significa que vamos por buen camino al probar el producto software.

7. RECOMENDACIONES:

- Aplicar futuramente TDD desde las primeras etapas del desarrollo para evitar problemas que pueden ser más difíciles de solucionarlos al futuro.

- Alcanzar el 100% de cobertura para que el producto software sea confiable además que el usuario se sienta conforme con la entrega que se le realizara.

8. BIBLIOGRAFÍA:

Angular Team. (2025). Testing - Angular official documentation. Recuperado de

<https://angular.io/guide/testing>.

Jasmine Team. (2025). Jasmine 4.6 Documentation. Recuperado de <https://jasmine.github.io/>.

Karma Team. (2025). Karma - Spectacular Test Runner for JavaScript. Recuperado de

<https://karma-runner.github.io/>.

Fowler, M. (2018). Test Driven Development (TDD). Recuperado de

<https://martinfowler.com/bliki/TestDrivenDevelopment.html>.

Enlace a git:

<https://github.com/Sebas8173/UniversityRepository/tree/main/Pruebas%20de%20software/U2/Lab%201>